

following example:

```
Array A = [1, 2]; Array B = [3]; K = 3;
```

In this case, the result should be `[[1,3], [2,3]]`, which consists of all possible pairs. Once you have confirmed that your pseudo-code handles all the corner cases, then code the same in your favourite programming language. If you are seeing any standard library functions, make sure that you understand the time and space complexities of the library calls you are making in your code. I will leave it to our readers to code up the actual solution and test it.

2. Here is another simple problem. You are given a two-dimensional matrix of size $(n \times n)$ consisting of integers. Each row of the matrix is sorted. Also, the first element of each row is larger than the last element of the previous row. Given a target value 't', write a function that searches the given matrix and returns 'true' if the value is present; else it returns 'false'.

As before, first let us look at the brute force approach. The simplest approach is to scan all the elements of the matrix, one by one, and look for the target value. This will result in a time complexity of $O(n^2)$ since we have to look at all the matrix elements. How can we do better? We are not using the information that the matrix has all its rows sorted. Not only that, this matrix has the additional property that each preceding row's last element is smaller than the succeeding row's first element. Essentially, instead of considering it as a $(n \times n)$ matrix, we can consider it as a single list of $(n \times n)$ elements where the rows are laid out, one after another. Immediately, we can think of using binary search since we have a sorted single dimensional list of integers and we need to find whether a value is present in it. How can we come up with an efficient implementation of this idea?

Explicitly creating a single dimensional copy of the two-dimensional matrix will end up in examining all elements. We know that the smallest element in the matrix is the top left hand corner element `A[0][0]`. The largest element is the bottom rightmost element `A[n-1][n-1]`. First, we compare the given value against each of these two elements. If the given value is smaller than `A[0][0]`, then the value is not present in the matrix. Similarly, if it is greater than `A[n-1][n-1]`, then too, it is not present. Now let us consider the first elements of all the rows. We need to find a Row Index `I` such that `A[i][0]` is smaller than target and `A[i+1][0]` is larger. Then we know that we need to examine only all the elements in Row 'I' to see if the value is present there.

Since Row 'I' is sorted, we can again do a binary search on Row 'I' to search for the value. Therefore, we only need two binary searches – one on the single

dimensional list consisting of all the first elements of all rows. And once we have identified the possible row, we do one more binary search on that specified row. Hence, the time complexity is $O(\log N)$ and the space complexity is $O(N)$ since we will have a copy of all the first elements of the rows in a single list. I will leave it to our readers to write up the code for this problem.

Let us consider a couple of other interview questions which are related to the problems we have discussed so far.

3. Consider a two-dimensional matrix A of size $(m \times n)$ containing integers (where 'm' is the number of rows and 'n' is the number of columns). In this problem, we are told that each row is sorted in ascending order. Also, each column is sorted in ascending order. The problem is to check whether a particular target value 't' is present in the matrix. If present, return 'true'; else return 'false'. While it looks similar to the problem we have discussed above, take note of what is different. In this case, we can't consider the entire matrix as a single ordered list, since there is no guarantee that the first element of each row is greater than or equal to the last element of the preceding row. So, our earlier technique will not work. Since we know that all the rows and all the columns are sorted, we can do binary search on all the rows (or on all the columns, respectively). This will result in a complexity of $O(n \log m)$, where we assume that $\min(m, n) = n$. Alternatively, if $\min(m, n) = m$, the time complexity will be $O(m \log n)$.

While this looks to be an efficient solution, the interviewer wants you to come up with a better solution, one that has a time complexity of $O(m+n)$. I will leave this problem for our readers to ponder over and will discuss the solution in next month's column.

Feel free to reach out to me over LinkedIn/email if you need any help in your new study journey. If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com.

I request our readers to follow the public health guidelines. Please don't worry if you are not able to focus fully on learning new technologies, programming, etc. These are difficult times. So first take care of your mental and physical health, help others wherever you can, and stay safe. Wishing all our readers happy coding until next month! Stay healthy and stay safe. **END** 🐧

By: Sandya Mannarswamy

The author is an expert in natural language processing and is currently working as an independent researcher. Her interests include natural language processing, machine learning and AI.